INFOSYS LIMITED · BHUBANESWAR DC · PMIS INTERNSHIP · LAB 4 & LAB 5

SOFTWARE REQUIREMENTS SPECIFICATION

NeoBank Platform — Sprint 2 — Budgeting, Bills & Rewards

Project	NeoBank Platform
Document	Software Requirements Specification (SRS)
Sprint	Sprint 2 — Budgeting, Bills & Rewards
Duration	10 Working Days (Days 11–20 of 60-Day Lifecycle)
Program	PMIS (Pradhan Mantri Internship Scheme) — Infosys Limited
Location	Bhubaneswar Development Centre — Lab 4 & Lab 5
Version	1.0 — Draft, Internal Review
Date	April 2026
Status	Active — Sprint In Progress
Previous Sprint	Milestone 1 & 2 — Authentication & Core Banking
Next Milestone	Milestone 5 — Loan Management

1. INTRODUCTION

The NeoBank Platform is an enterprise-grade digital banking solution developed under the PMIS (Pradhan Mantri Internship Scheme) Internship Program at the Infosys Bhubaneswar Development Centre, Lab 4 and Lab 5. This document is the authoritative Software Requirements Specification (SRS) for Sprint 2, covering Days 11 through 20 of the 60-day development lifecycle.

With the core authentication and banking ledger fully established in Sprint 1, this sprint builds the actionable financial layer of NeoBank. Sprint 2 delivers automated expense tracking through budget management, proactive bill management with reminder workflows, and a user reward points system — together forming a comprehensive personal finance experience on the platform.

1.1 Purpose

This SRS defines the complete set of functional requirements, system behaviour, business rules, integration strategy, quality assurance plan, and sprint-level execution roadmap for NeoBank Sprint 2. It serves as the binding technical reference for all interns, mentors, and stakeholders in Lab 4 and Lab 5.

1.2 Scope of Sprint 2

Sprint 2 is scoped to the personal finance and engagement layer. Deliverables are limited to:

- FR-3: Budgeting and Expense Management — monthly budget creation, category-wise expense tracking, utilization monitoring
- FR-4: Bills, Rewards, and Notifications — bill creation, payment status tracking, bill reminders, and reward points management
- Database schema extension: provisioning Budget, Bill, and Reward tables via DDL scripts
- Frontend-backend integration: Angular components wired to Spring Boot REST APIs with JWT-secured requests
- Unit and end-to-end testing of all sprint deliverables
- Swagger/OpenAPI documentation for all new endpoints

1.3 Project Context

Internship Program	PMIS (Pradhan Mantri Internship Scheme)
Organisation	Infosys Limited
Development Centre	Bhubaneswar DC
Labs	Lab 4 & Lab 5
Project	NeoBank Platform
Sprint Duration	10 Working Days (Days 11–20)
Total Cycle	60 Working Days
Previous Sprint	Sprint 1 — Authentication & Core Banking (Days 1–10)
Next Sprint	Milestone 5 — Loan Management

1.4 Definitions & Abbreviations

TERM	DEFINITION
SRS	Software Requirements Specification — this document
FR	Functional Requirement — a categorised set of deliverable features
JWT	JSON Web Token — signed stateless token used for API authentication
RBAC	Role-Based Access Control — access restrictions based on assigned roles
DDL	Data Definition Language — SQL scripts to create and extend database schema
API	Application Programming Interface — RESTful HTTP endpoints
ORM	Object-Relational Mapping — Spring Data JPA maps Java entities to MySQL
DTO	Data Transfer Object — decouples API payload structure from JPA entity
PMIS	Pradhan Mantri Internship Scheme — the government internship program hosted by Infosys
DC	Development Centre — Infosys Bhubaneswar facility

2. FUNCTIONAL REQUIREMENTS OVERVIEW

Sprint 2 delivers two core Functional Requirement modules that extend NeoBank beyond ledger management into personal finance. These modules depend on the JWT authentication and account infrastructure completed in Sprint 1, and must be fully functional and tested before sprint sign-off.

2.1 FR-3: Budgeting and Expense Management

This module provides users with the ability to plan and monitor their monthly finances. It introduces budget allocations per spending category, maps account transactions to those categories, and computes real-time utilization metrics.

2.1.1 Budget Creation

- Authenticated users create monthly budgets via POST /api/budgets
- Each budget entry is bound to a specific category (e.g., Groceries, Utilities, Rent, Entertainment, Transfer)
- Budget limit amount must be strictly greater than zero — zero or negative values rejected with HTTP 400 Bad Request
- Only one active budget per category per month per user is permitted — duplicate entries rejected with HTTP 409 Conflict
- Budget ownership is resolved exclusively from the JWT payload — never from the request body
- Successful creation returns HTTP 201 Created with the full budget resource

2.1.2 Budget Summary and Tracking

- Users retrieve their budget summary via GET /api/budgets/{userId}/{month}
- Response includes each category's allocated limit, total spent, remaining allowance, and utilization percentage
- Expenses are derived by aggregating transactions from the accounts module, mapped to predefined valid categories
- Only authenticated users may view their own budget data — cross-user access rejected with HTTP 403 Forbidden
- Month parameter accepted in YYYY-MM format

2.2 FR-4: Bills, Rewards, and Notifications

FR-4 introduces recurring financial obligations and a user engagement layer. It enables bill scheduling with reminders, payment status lifecycle management, and a reward points system tied to platform activity.

2.2.1 Bill Management

- Authenticated users create bill records via POST /api/bills
- Each bill includes biller name, amount, due date, and payment status (PENDING / PAID / OVERDUE)
- Due date must be a future date — past due dates rejected with HTTP 400 Bad Request
- Duplicate bills for the same biller and month are strictly prohibited — HTTP 409 Conflict returned
- System generates reminder flags for bills approaching their due date within a configurable threshold (default: 3 days)
- Payment status can be updated via PATCH /api/bills/{id}/status

2.2.2 Rewards System

- Users retrieve their reward point balance via GET /api/rewards/{userId}
- Reward points are accrued through platform activity (e.g., transactions, bill payments)

- Points balance must always remain non-negative — no operation may result in a negative balance
- Reward calculation logic executes on the backend; the Angular UI displays the current balance and accrual history
- Cross-user reward access is prohibited — JWT userId is validated against the requested resource

3. BUSINESS & VALIDATION RULES

The following rules are hard constraints enforced at the backend API layer, independent of any frontend validation. Any API response that violates these rules is classified as a defect.

FR-3 — BUDGETING RULES

REF	CONSTRAINT
BR-01	Budget limit amount must be strictly greater than zero. Zero or negative amounts return HTTP 400 Bad Request with a descriptive error body.
BR-02	Only one budget entry is permitted per category per month per user. Duplicate registration attempts return HTTP 409 Conflict.
BR-03	Expenses must be mapped to valid, predefined categories only. Requests referencing undefined categories return HTTP 400 Bad Request.
BR-04	Budget ownership must be resolved from the Spring SecurityContext JWT payload — the userId must never be accepted from the HTTP request body.
BR-05	Cross-user budget access is strictly prohibited. Requests for another user's budget data return HTTP 403 Forbidden.
BR-06	Month parameter must conform to YYYY-MM format. Malformed month values return HTTP 400 Bad Request.

FR-4 — BILLS & REWARDS RULES

REF	CONSTRAINT
BR-01	A bill's due date cannot be set in the past. Requests with a past due date return HTTP 400 Bad Request before any database write occurs.
BR-02	Duplicate bills for the same biller name within the same calendar month are strictly prohibited. Such requests return HTTP 409 Conflict.
BR-03	Reward point balances must always remain non-negative. Any operation that would reduce the balance below zero must be rejected with HTTP 422 Unprocessable Entity.
BR-04	Payment status transitions are restricted to valid lifecycle states: PENDING → PAID or PENDING → OVERDUE. Invalid transitions return HTTP 400 Bad Request.
BR-05	Bill and reward resources are user-scoped. Cross-user access is rejected with HTTP 403 Forbidden — resource ownership validated against the JWT payload.
BR-06	All secured bill and reward endpoints require a valid, non-expired JWT Bearer token in the Authorization header. Missing or invalid tokens return HTTP 401 Unauthorized.

4. INFRASTRUCTURE & ARCHITECTURE

Sprint 2 extends the same modern, loosely-coupled, three-tier full-stack architecture established in Sprint 1. No architectural changes are introduced; new modules are added as extensions to the existing layer structure.

4.1 Technology Stack

LAYER	TECHNOLOGY	DETAILS
Frontend	Angular	TypeScript SPA; Angular CLI; Reactive Forms; HTTP Interceptors; Route Guards; Charting libraries for utilization visuals
Backend	Spring Boot	Java REST API; Spring Security; Spring Data JPA; Hibernate ORM; Maven; @Transactional for atomic operations
Database	MySQL 8.0+	Relational RDBMS; InnoDB engine; DDL scripts extending Sprint 1 schema; FK constraints to accounts and users tables
Security	JWT + BCrypt	Stateless token auth inherited from Sprint 1; Bearer scheme; all new endpoints secured via JwtAuthFilter
Version Control	Git	Feature-branch workflow; Sprint 2 branches from develop; all work on feature/sprint2-* branches
API Docs	Swagger/OpenAPI	springdoc-openapi extended with new FR-3 and FR-4 endpoints; interactive UI at /swagger-ui.html
IDE	IntelliJ + VSCode	IntelliJ for Spring Boot; VS Code with Angular Language Service for frontend

4.2 Database Schema Extensions

Three new tables are provisioned on Day 11 via DDL scripts, extending the existing Sprint 1 schema. Foreign key constraints are enforced with the InnoDB engine.

4.2.1 budgets

- id — BIGINT, PK, AUTO_INCREMENT
- user_id — BIGINT, FK → users(id), ON DELETE CASCADE
- category — ENUM('GROCERIES', 'UTILITIES', 'RENT', 'ENTERTAINMENT', 'TRANSFER', 'OTHER'), NOT NULL
- budget_month — DATE, NOT NULL (stored as first day of the month, e.g., 2026-04-01)
- limit_amount — DECIMAL(15,2), NOT NULL, CHECK (limit_amount > 0)
- created_at — TIMESTAMP, DEFAULT CURRENT_TIMESTAMP
- UNIQUE KEY uq_budget (user_id, category, budget_month) — enforces one budget per category per month

4.2.2 bills

- id — BIGINT, PK, AUTO_INCREMENT
- user_id — BIGINT, FK → users(id), ON DELETE CASCADE
- biller_name — VARCHAR(255), NOT NULL
- amount — DECIMAL(15,2), NOT NULL, CHECK (amount > 0)
- due_date — DATE, NOT NULL
- status — ENUM('PENDING', 'PAID', 'OVERDUE'), DEFAULT 'PENDING'
- created_at — TIMESTAMP, DEFAULT CURRENT_TIMESTAMP

- UNIQUE KEY uq_bill (user_id, biller_name, YEAR(due_date), MONTH(due_date)) — enforces no duplicate bills per biller per month

4.2.3 rewards

- id — BIGINT, PK, AUTO_INCREMENT
- user_id — BIGINT, FK → users(id), ON DELETE CASCADE, UNIQUE
- points_balance — INT, NOT NULL, DEFAULT 0, CHECK (points_balance >= 0)
- last_updated — TIMESTAMP, DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP

5. INSTALLATION & SETUP — DAY 11

Day 11 is dedicated to Sprint 2 environment extension and data modeling. Both sessions must be completed before Day 12 development begins. A fully extended schema is a hard prerequisite for all subsequent sprint activity.

5.1 Session 1: Sprint 2 Branch & Frontend Dependencies

Git Branch Setup

- Create a new Git branch from develop: git checkout -b feature/sprint2-budgeting-bills-rewards
- Push the branch to remote: git push -u origin feature/sprint2-budgeting-bills-rewards
- Share branch access with all team members and the PMIS mentor

Angular Dependencies

- Install charting library for budget utilization visuals: npm install ng2-charts chart.js (or equivalent)
- Verify chart.js installation: import { NgChartsModule } in AppModule
- Install any masonry/card layout libraries required for dashboard aesthetics
- Run ng serve to confirm the Angular application starts without errors

5.2 Session 2: Database Schema Extension

DDL Execution Order

- 04_create_budgets_table.sql — includes UNIQUE constraint on (user_id, category, budget_month)
- 05_create_bills_table.sql — includes UNIQUE constraint on (user_id, biller_name, month/year of due_date)
- 06_create_rewards_table.sql — one row per user, points_balance >= 0 enforced
- Verify: SHOW TABLES; confirms all 6 tables present
- Verify: DESCRIBE budgets; DESCRIBE bills; DESCRIBE rewards; — validate all columns

Spring Boot Entity & DTO Setup

- Create Budget, Bill, Reward JPA entities with @Entity, @Table, @Id annotations
- Create BudgetRepository, BillRepository, RewardRepository extending JpaRepository
- Create corresponding DTOs: BudgetRequestDTO, BudgetSummaryDTO, BillRequestDTO, BillResponseDTO, RewardDTO
- Configure Spring Boot to validate schema against existing DDL with hibernate.ddl-auto=validate

6. SPRINT 2 EXECUTION PLAN

The 10-day sprint enables parallel backend and frontend development. An integration session on Day 18 bridges both layers. Each session is approximately 3–4 hours of focused work.

6.1 Sprint Schedule

DAY	PHASE	SESSION 1	SESSION 2
11	Env Setup	Create Sprint 2 Git branch; install Angular charting dependencies	Execute DDL scripts for Budget, Bill, Reward tables; create JPA entities and DTOs
12	Budget Backend	POST /api/budgets — budget creation with ownership from JWT SecurityContext	Enforce limit > 0 and single-budget-per-category-per-month validation
13	Budget Tracking	GET /api/budgets/{userId}/{month} — summary with utilization calculation	Map account transactions to budget categories; aggregate spent amounts
14	Budget UI	Angular budget creation form — responsive, category dropdown, limit input	Dashboard components: category-wise bars, utilization percentage, remaining allowance
15	Bills Backend	POST /api/bills — bill creation with future-date validation	Duplicate biller-month check; PATCH /api/bills/{id}/status for payment updates
16	Rewards Backend	GET /api/rewards/{userId} — fetch reward balance and accrual history	Reward calculation logic; non-negative balance enforcement; accrual triggers
17	Bills & Rewards UI	Angular bill creation, listing, and payment status update components	Angular reward points display; points balance, accrual history, reminder badges
18	Integration	Wire Angular budget UI to Spring Boot REST endpoints; validate DTO mapping	Connect bills and rewards Angular components to backend APIs; verify JWT propagation
19	QA	Unit tests: budget rules, bill validations, reward constraints	End-to-end testing: Budget Tracking and Bill Management module flows
20	Wrap-Up	Update Swagger/OpenAPI documentation for all new endpoints	Final UI review; QnA session; sprint sign-off; prep for Loan Management sprint

6.2 Day 12: Budget Creation Engine

Session 1 — Backend: POST /api/budgets

- Create Budget JPA entity with @Entity, @Table(uniqueConstraints) for (user_id, category, budget_month)
- BudgetRepository extends JpaRepository<Budget, Long>; add findByUserIdAndCategoryAndBudgetMonth()
- BudgetService.create(): extract userId from SecurityContextHolder → validate limit > 0 → check duplicate → save → return DTO
- BudgetController @PostMapping("/api/budgets") → 201 Created or 400/409

Session 2 — Backend: Validation

- Validate limit_amount > 0 with @Positive or manual check; return HTTP 400 on failure
- Query for existing budget: same user_id + category + month → HTTP 409 if present

- Category validated against ENUM values; unrecognised category → HTTP 400
- Write unit tests for all three validation paths before Day 14

6.3 Day 13: Budget Summary & Tracking

Session 1 — Backend: GET /api/budgets/{userId}/{month}

- Parse month parameter as YYYY-MM; validate format → HTTP 400 if invalid
- Fetch all budgets for the authenticated user and specified month
- For each budget category, aggregate transactions from accounts owned by the user within the same month
- Compute utilization: spent / limit_amount * 100; return as BudgetSummaryDTO

Session 2 — Backend: Category Mapping

- Transaction.description is matched to budget categories via a configurable keyword map
- Transactions without a matching category are classified as 'OTHER'
- @Transactional(readOnly = true) applied to all read operations for performance

6.4 Day 14: Budgeting UI

Session 1 — Budget Creation Form

- ng generate component budget/create
- Reactive FormGroup: category (dropdown), budgetMonth (date picker), limitAmount (numeric)
- Validators: required, min(0.01) for amount, valid month format
- BudgetService.create() via HttpClient; display 201 success or field-level validation errors

Session 2 — Budget Dashboard

- ng generate component budget/dashboard
- Display each category with progress bar showing spent vs. limit (utilization %)
- Colour coding: under 75% → green, 75–99% → amber, 100%+ → red
- Integrate chart.js doughnut or bar chart for visual budget breakdown

6.5 Day 15: Bill Management Engine

Session 1 — Backend: POST /api/bills

- Create Bill JPA entity; BillRepository with findByUserIdAndBillerNameAndDueDateBetween()
- BillService.create(): validate due_date is in the future → check duplicate biller/month → save → return DTO
- BillController @PostMapping("/api/bills") → 201 Created or 400/409

Session 2 — Backend: Status Updates & Duplicates

- PATCH /api/bills/{id}/status: validate ownership → validate status transition → update → return updated resource
- Duplicate check: query for existing bill with same user_id + biller_name within the same calendar month
- Reminder flag: on GET /api/bills, set a boolean remindMe = true if due_date is within 3 days

6.6 Day 16: Rewards System

Session 1 — Backend: GET /api/rewards/{userId}

- Create Reward JPA entity (one-to-one with User); RewardRepository

- RewardService.getBalance(): validate JWT userId matches path userId → fetch and return RewardDTO
- If no reward record exists for user, auto-create with points_balance = 0

Session 2 — Backend: Reward Calculation

- Reward accrual triggered on: successful bill payment (PENDING → PAID transition), creation of new transactions above a threshold
- Points deduction operations check points_balance >= deduction_amount before proceeding
- All point mutations wrapped in @Transactional to prevent race conditions

6.7 Day 17: Bills & Rewards UI

Session 1 — Bills UI

- ng generate component bills/list — display biller, amount, due date, status, reminder badge
- ng generate component bills/create — form with biller name, amount, due date fields
- Status update button: MARK AS PAID triggers PATCH call; updates status badge in real time
- Overdue styling: bills past due date highlighted in red via Angular class binding

Session 2 — Rewards UI

- ng generate component rewards/dashboard — display current points balance prominently
- Accrual history list: date, description, points earned/spent
- Points balance update reflected in header/navigation bar via shared Angular service

7. API REFERENCE

All secured endpoints require a valid JWT Bearer token in the Authorization header. Endpoints inherited from Sprint 1 remain unchanged and are not repeated here.

FR-3 — BUDGETING ENDPOINTS

ENDPOINT	METHOD	DAY	AUTH	DESCRIPTION
POST /api/budgets	POST	Day 12	JWT	Create monthly budget for a category. Enforces limit > 0 and uniqueness. Returns 201.
GET /api/budgets/{userId}/{month}	GET	Day 13	JWT	Fetch budget summary with utilization stats. 403 if userId mismatches JWT. Month: YYYY-MM.
GET /api/budgets	GET	Day 13	JWT	List all budgets for the authenticated user across all months.
DELETE /api/budgets/{id}	DELETE	Day 12	JWT	Delete a specific budget entry. Ownership validated from JWT. Returns 204.

FR-4 — BILLS, REWARDS & NOTIFICATIONS ENDPOINTS

ENDPOINT	METHOD	DAY	AUTH	DESCRIPTION
POST /api/bills	POST	Day 15	JWT	Create a new bill record. Validates future due date and biller uniqueness per month. Returns 201.
GET /api/bills	GET	Day 15	JWT	List all bills for the authenticated user. Includes remindMe flag for bills due within 3 days.
GET /api/bills/{id}	GET	Day 15	JWT	Fetch a specific bill. 403 if not owner; 404 if not found.
PATCH /api/bills/{id}/status	PATCH	Day 15	JWT	Update bill payment status (PENDING → PAID or OVERDUE). Validates ownership and transition.
GET /api/rewards/{userId}	GET	Day 16	JWT	Fetch reward points balance and accrual history. 403 if userId mismatches JWT.

8. INTEGRATION STRATEGY

The integration session on Day 18 bridges Angular and Spring Boot for all Sprint 2 modules. This session validates end-to-end data flow from the user interface through the API layer to MySQL and back, building on the JWT infrastructure verified in Sprint 1's Day 5 and Day 10 integration sessions.

8.1 Day 18 — Sprint 2 Full Integration

Session 1 — Budgeting Integration

- Wire Angular BudgetService to POST /api/budgets and GET /api/budgets endpoints
- Confirm BudgetRequestDTO fields map correctly between Angular FormGroup and Spring Boot request body
- Verify JWT from sessionStorage is attached by JwtInterceptor on all budget API calls
- Test utilization calculation end-to-end: create transactions in Sprint 1 accounts → verify budget summary reflects correct spent amounts
- Validate error rendering: duplicate budget category → 409 displayed in UI; negative limit → 400 displayed

Session 2 — Bills & Rewards Integration

- Connect Angular BillService to POST /api/bills and GET /api/bills endpoints
- Wire reward dashboard to GET /api/rewards/{userId}; verify balance reflects backend state
- Test PATCH /api/bills/{id}/status from Angular: mark as paid → confirm status badge updates without page reload
- Validate reminder badge: create a bill due within 3 days → confirm remindMe = true in API response → badge appears in Angular UI
- Cross-verify reward points in Angular against SELECT points_balance FROM rewards WHERE user_id = ? in MySQL
- Test JWT expiry scenario: expired token → 401 → Angular AuthGuard redirects to /login

9. DATA SEEDING

SQL seeding scripts populate the new Sprint 2 tables after schema creation, enabling integration and QA sessions to proceed with consistent, representative test data. Sprint 1 seed data is preserved and extended.

ENTITY	COUNT	SEED DETAILS
Budget Records	9+	3 budgets per customer user across 3 categories (Groceries, Utilities, Rent) for the current month; varying limit amounts
Bill Records	6+	Mix of PENDING, PAID, and OVERDUE statuses; at least 1 bill per customer due within 3 days to test reminder functionality
Reward Records	3	One reward record per customer user; initial points_balance of 0 for fresh accounts
Budget Categories	N/A	Groceries, Utilities, Rent, Entertainment, Transfer, Other — all available in dropdown seeded via ENUM definition
Transaction Linkage	N/A	Ensure Sprint 1 sample transactions include descriptions mappable to at least 2 budget categories for utilization testing

10. QUALITY ASSURANCE & TESTING

QA is integrated into Day 19. Testing covers both the budgeting module and the bills and rewards module. All tests must pass before sprint sign-off on Day 20.

DAY 19 — FR-3 BUDGETING QA

Unit Tests

- Budget creation — negative limit: assert HTTP 400 returned; zero amount: assert HTTP 400
- Budget creation — duplicate category/month: assert HTTP 409 Conflict with descriptive error body
- Budget creation — invalid category: assert HTTP 400 Bad Request
- Budget summary — correct utilization: create budget of 1000, add 400 in transactions → assert utilization = 40%
- Budget summary — cross-user access: different JWT userId → assert HTTP 403 Forbidden
- Budget summary — invalid month format: assert HTTP 400 Bad Request
- Balance accuracy: assert spent = sum of transactions in category within month

End-to-End Tests

- Login → Create Budget (Groceries, limit 2000) → perform 3 transactions in Groceries → view budget summary → verify utilization matches
- Duplicate budget attempt from Angular UI: verify conflict error message displayed correctly
- Verify Angular progress bar colour: 60% utilized → green; 80% → amber; 110% → red

DAY 19 — FR-4 BILLS & REWARDS QA

Unit Tests

- Bill creation — past due date: assert HTTP 400 Bad Request before any database write
- Bill creation — duplicate biller/month: assert HTTP 409 Conflict
- Bill status update — invalid transition: assert HTTP 400 Bad Request
- Bill reminder flag: create bill with due_date = today + 2 days → assert remindMe = true in GET response
- Reward balance — non-negative enforcement: simulate deduction exceeding balance → assert HTTP 422
- Reward balance — cross-user access: different JWT userId → assert HTTP 403 Forbidden
- Bill ownership: different user JWT accessing another user's bill → assert HTTP 403

End-to-End Tests

- Login → Create Bill → mark as PAID → verify status badge updates in Angular without page reload
- Create bill due in 2 days → verify reminder badge appears in Angular bills list
- Verify overdue styling: create bill with status OVERDUE → confirm red highlighting in Angular UI
- Cross-verify reward points balance in Angular against SELECT points_balance FROM rewards WHERE user_id = ? in MySQL

11. DOCUMENTATION & SPRINT WRAP-UP

Day 20 concludes with a documentation and sign-off session. All deliverables below must be committed to Git before the sprint is considered closed.

11.1 API Documentation

- Extend springdoc-openapi annotations to cover all new FR-3 and FR-4 endpoints
- Annotate BudgetController, BillController, RewardController with @Operation and @ApiResponse
- Annotate all new DTOs with @Schema annotations for field descriptions
- Verify Swagger UI at http://localhost:8080/swagger-ui.html reflects all 9 new endpoints
- Export OpenAPI JSON: http://localhost:8080/v3/api-docs → commit as docs/openapi-sprint2.json

11.2 Frontend Documentation

- Document all new Angular components: purpose, inputs/outputs, services injected, routes
- Document BudgetService, BillService, RewardService: methods, HTTP calls, error handling
- Update routing module documentation to include /budget, /bills, /rewards routes
- Update README.md in neobank-frontend with Sprint 2 features and any new environment variables

11.3 Sprint Review & Sign-Off

- Live demonstration of all Sprint 2 deliverables against acceptance criteria in this SRS
- Code review of both repositories with PMIS mentor — focus on business rule enforcement and JWT security
- QnA session for Lab 4 and Lab 5 intern teams
- Formal mentor sign-off upon acceptance of all deliverables
- Workspace preparation for Milestone 5: Loan Management

12. ACCEPTANCE CRITERIA

All criteria below constitute the Definition of Done for Sprint 2. Every item must be met before the sprint can be signed off.

FR-3 — BUDGETING & EXPENSE MANAGEMENT

#	CRITERION	VERIFIED BY
AC01	Budget creation API enforces limit > 0 and single-budget-per-category-per-month constraints	Unit Test
AC02	Budget summary API returns correct utilization percentages based on linked transaction data	Unit Test + Postman
AC03	Budget ownership resolved exclusively from JWT payload — userId never accepted from request body	Unit Test
AC04	Cross-user budget access rejected with HTTP 403 Forbidden	Unit Test
AC05	Angular budget creation form functional with category dropdown and validation messages	E2E Test
AC06	Angular budget dashboard displays utilization bars with correct colour coding (green/amber/red)	E2E Test

FR-4 — BILLS, REWARDS & NOTIFICATIONS

#	CRITERION	VERIFIED BY
AC07	Bill creation API rejects past due dates with HTTP 400 and duplicate biller/month with HTTP 409	Unit Test
AC08	Bill status update enforces valid lifecycle transitions only (PENDING → PAID / OVERDUE)	Unit Test
AC09	Reminder flag (remindMe = true) correctly set for bills due within 3 days in GET /api/bills response	Unit Test + Postman
AC10	Reward points balance remains non-negative under all operations	Unit Test
AC11	Angular bills UI functional: creation, listing, status update, overdue styling, and reminder badges	E2E Test
AC12	Angular rewards dashboard displays current balance matching MySQL rewards table	E2E Test

GENERAL QUALITY

#	CRITERION	VERIFIED BY
AC13	All unit tests pass with zero critical failures across FR-3 and FR-4	Test Suite
AC14	Swagger UI accessible; all 9 new endpoints documented with request/response schemas	Manual Review
AC15	Git repositories clean: Sprint 2 feature branches merged, commit messages descriptive	Mentor Review

#	CRITERION	VERIFIED BY
AC16	No hardcoded credentials; code follows Infosys formatting standards	Code Review
AC17	README.md files updated in both repos reflecting Sprint 2 features and setup changes	Mentor Review